

Stop clicking, start prompting!

Visual Studio Code and Agentic AI for Power Platform builders

Nick Doelman

Microsoft MVP & MCT

Baltic Summit 2026 · Gdynia

SPONSORS



COMMUNITIES & PARTNERS



ABOUT YOUR SPEAKER



Nick Doelman

I help companies build and adopt intelligent business solutions.

Microsoft MVP (8x) · MCT · Solution architect · Trainer · Advisor
15+ years on Dataverse, Power Pages and model-driven apps



Microsoft MVP (8x)



Power Platform BOOST



Canada Powerlifting

nickdoelman.com

Blog, sessions, podcast and everything I publish. All of it lives there.

readyxrm.blog · [ReadyBMS](https://ReadyBMS.com) · Ottawa

TAKEAWAYS

- Visual Studio Code, set up right
- Agents that plan and self-check
- Dataverse and Power Platform skills
- A ticketing app, built by prompt
- Where it saves time, where it bites

#GrowBetterTogether

Unleash the power of Microsoft solutions

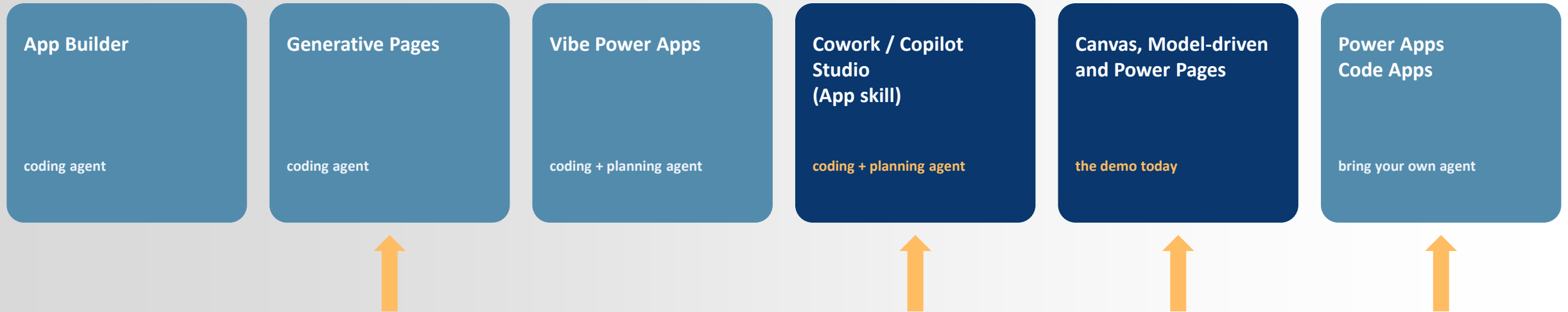
EVERYTHING BUILDS APPS NOW



AI-ASSISTED DEVELOPMENT IN POWER PLATFORM AND MICROSOFT 365

LOW CONTROL

HIGH CONTROL



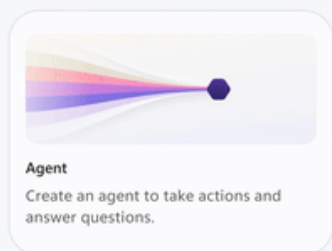
Agentic Tools + Skills

GitHub Copilot · Claude Code · Codex

Same skills, same workflow. What changes is how much of the result you own.

PREVIEW: BUILDING APPS IN COWORK AND COPILOT

Hey



Describe it. The platform builds it.

There is a second path being built, and it is not the one in this session.

Plain language in, a working agent or app out.

Governance is in the platform

Not a set of constructs you assemble yourself afterwards.

Lifecycle comes with it

Publishing, sharing and versioning come with the surface.

Built for AI apps first

Agents, actions and grounding are the primitives, not a retrofit.

[Blog home](#) > Build where you want, run with confidence: Now Microsoft hosts and manages the code created by Copilot

Search the blog



[Feature releases](#) • September 25 • 5 min read

Build where you want, run with confidence: Now Microsoft hosts and manages the code created by Copilot

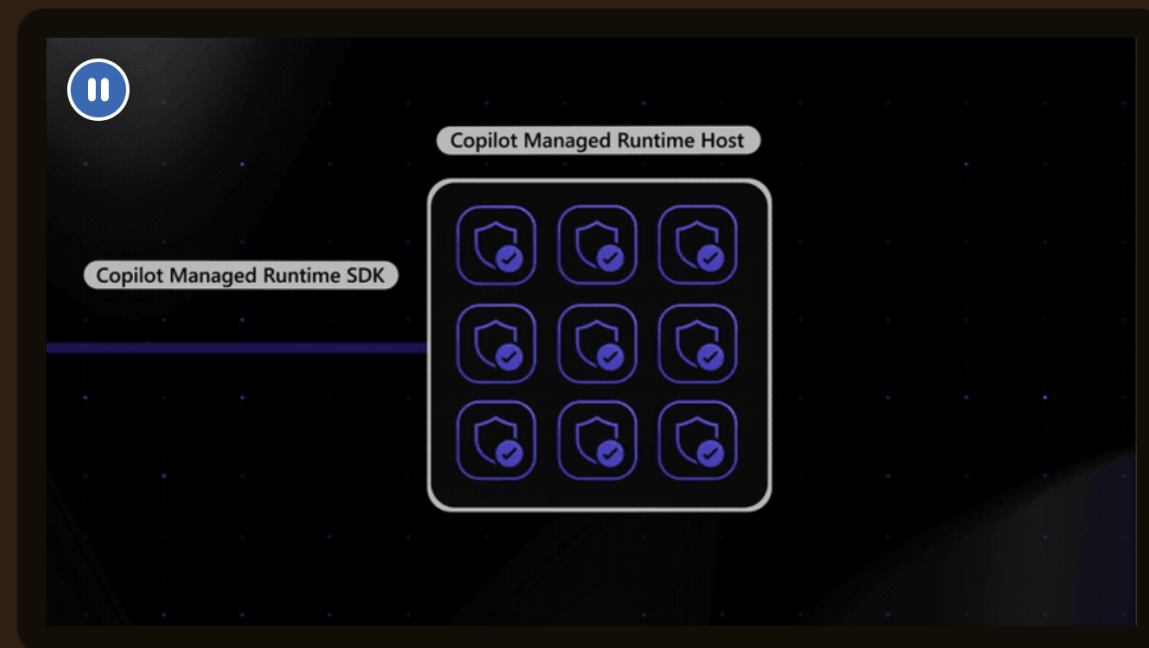
By [David Blyth](#), Vice President and General Manager, Managed Apps and Agents

Listen to this post



0:00 / 0:00

1X



SHARE



CONTENT TYPES

Feature releases

News

TOPICS

Code is becoming easier for anyone to create with AI. Apps in Copilot Cowork and the new Copilot Code experience bring that creation directly into Copilot, helping people move from an idea to working apps and code in the same place they already get work done.

Functioning code is only the first step. The work that follows includes provisioning and securing cloud resources, configuring identity and security

APPS IN COWORK AND COPILOT

Very early preview

- It is moving fast and it is not finished. What you see in a demo today is not what ships.

Not solution-aware

- No solutions. It is Git-driven, versioned and promoted the way traditional source control works, not the Power Platform constructs you know.

Consumption licensing

- Priced on what it consumes rather than per user. Whether the market accepts that is an open question.

So we build here today

- Dataverse, solutions, environments. The thing you can hand to somebody else and support.

VISUAL STUDIO CODE IS A MAKER TOOL



YOU ARE ALREADY A DEVELOPER

THE OBJECTION

Visual Studio Code is a developer tool.

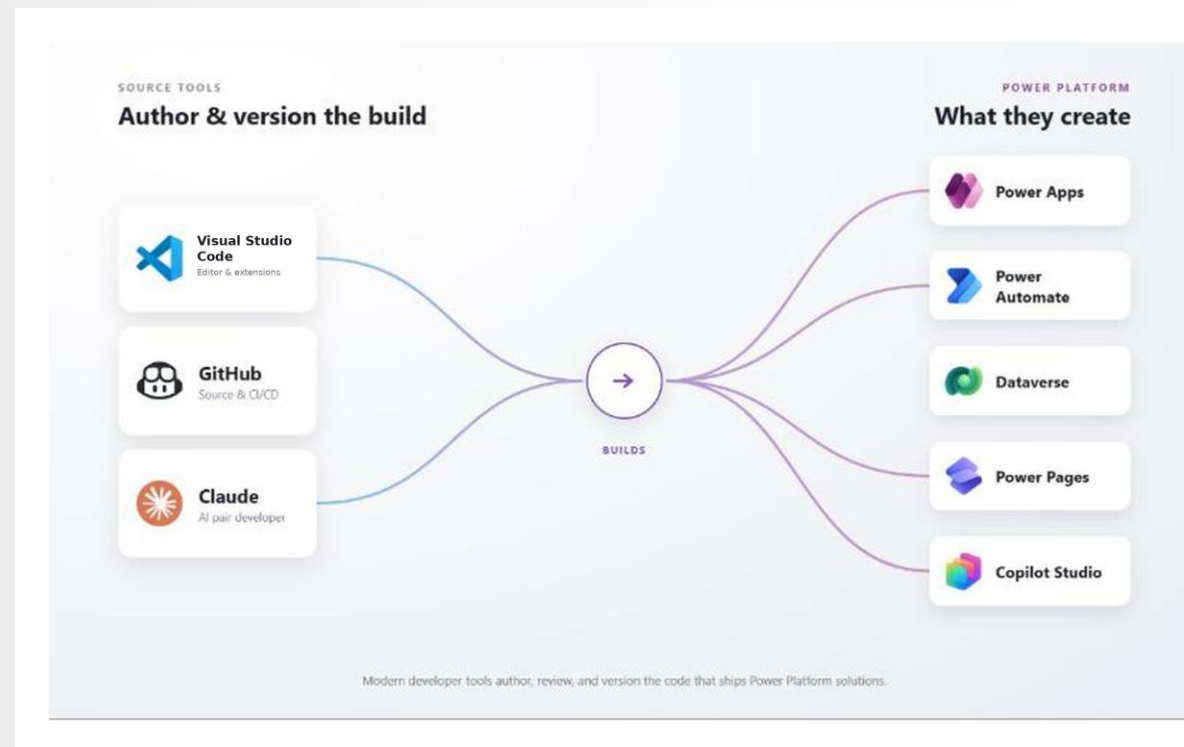
I am just a maker. The command line frightens me.

THE ANSWER

If you build applications, you are as much a developer as anyone.

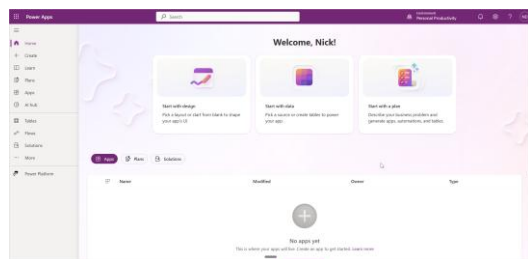
Visual Studio Code is a text editor with extensions. Treat it as a maker tool.

And the best form of low code is natural language, which is what developers have been chasing for decades.



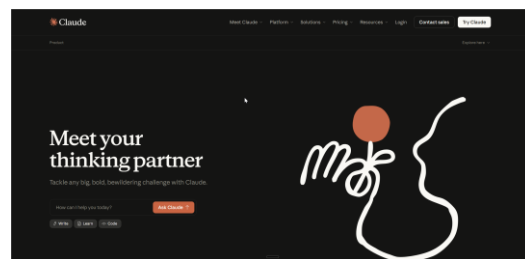
The same tools, pointed at the Power Platform.

WHAT YOU NEED BEFORE YOU START



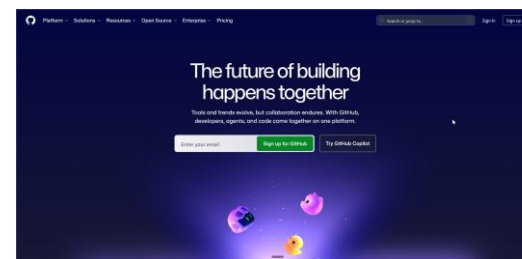
A Power Platform tenant

Nothing here replaces it. You still need an environment to build in.



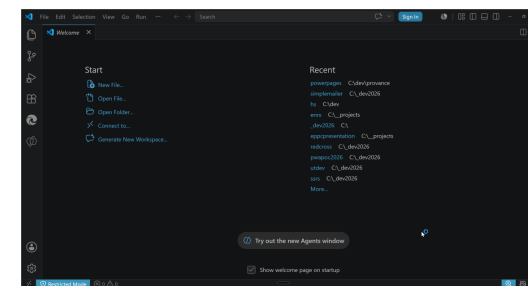
An agentic coding tool

Claude Code today. GitHub Copilot or Codex work the same way.



A GitHub account

Free is fine. This is where you save progress and track history.

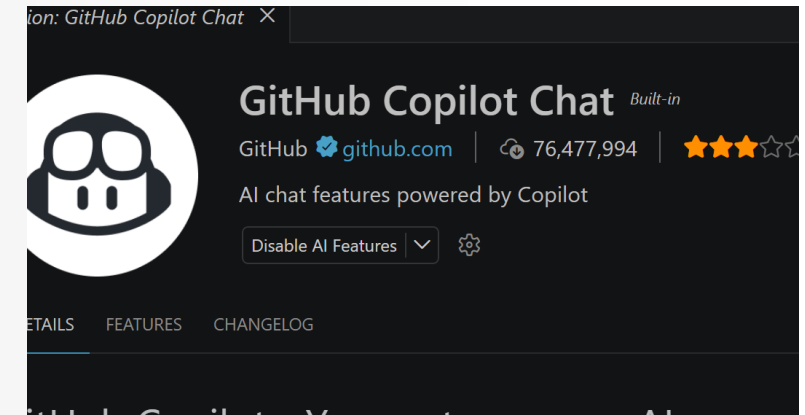
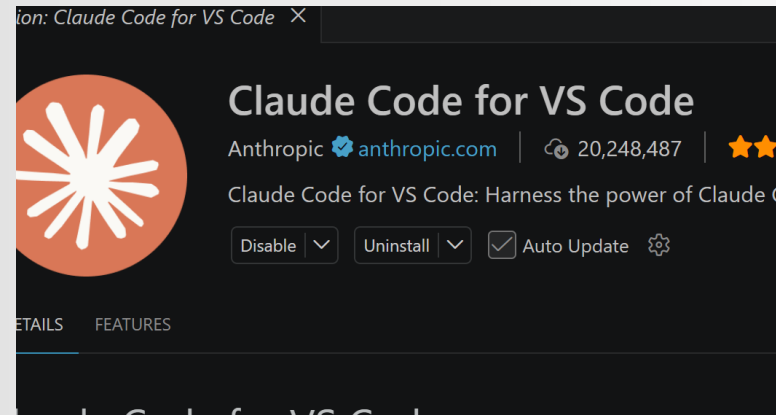
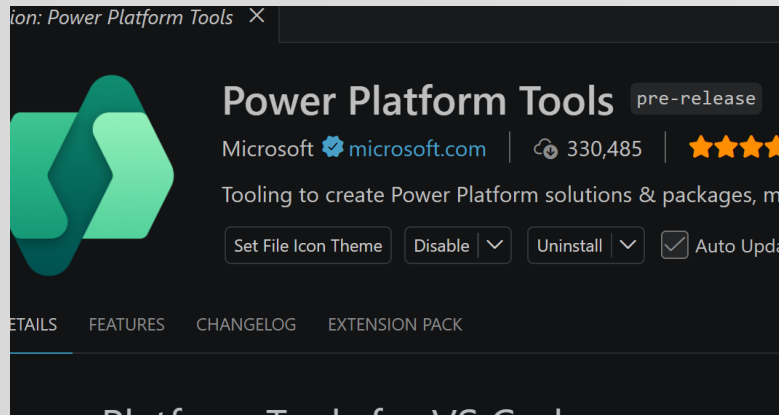


Visual Studio Code

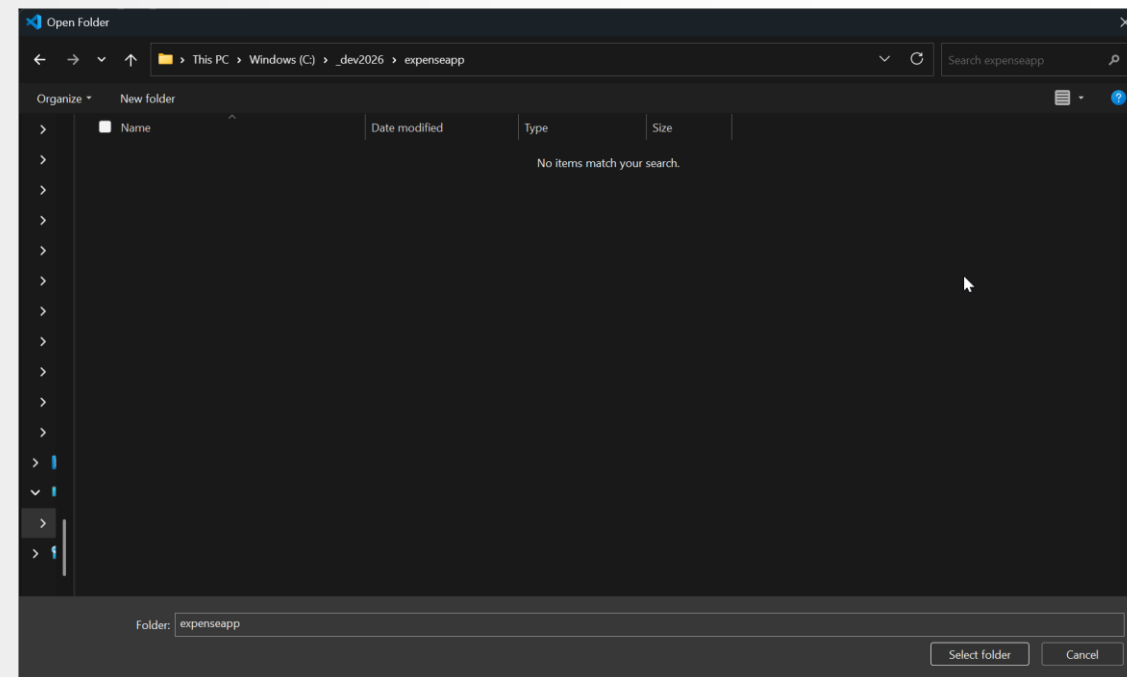
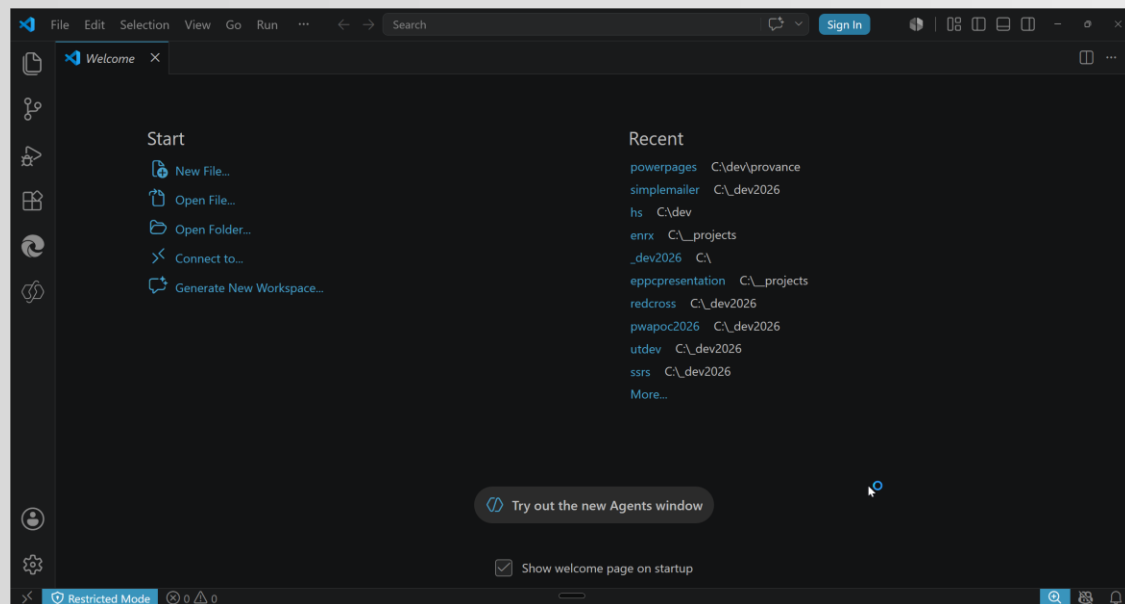
Free from Microsoft. Windows, Mac and Linux. The hub for the rest.

Four sign-ups and one download. That is the barrier to entry.

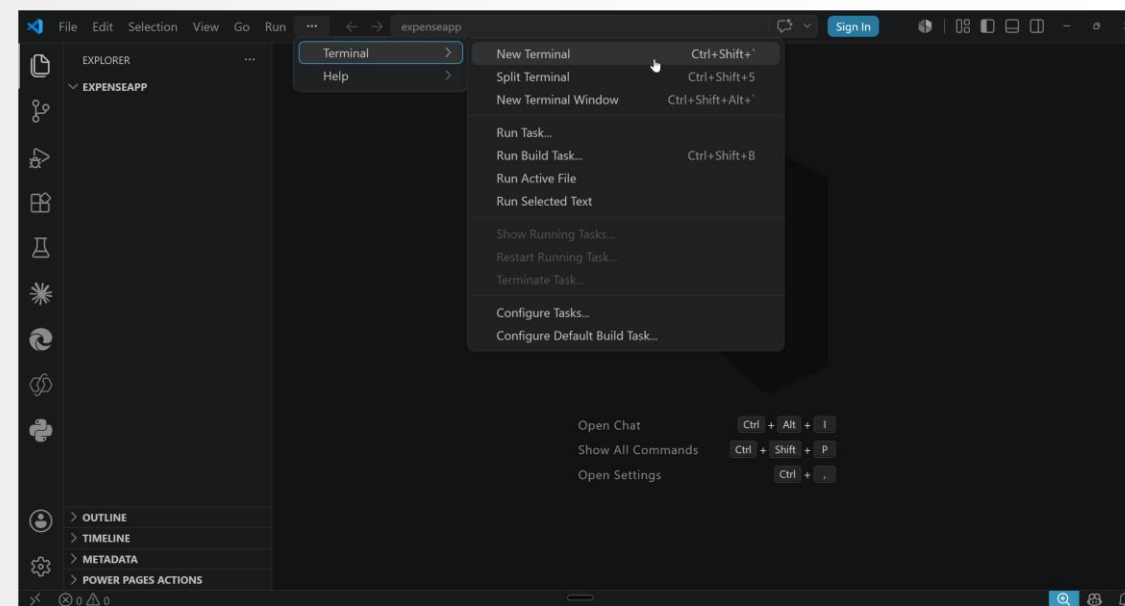
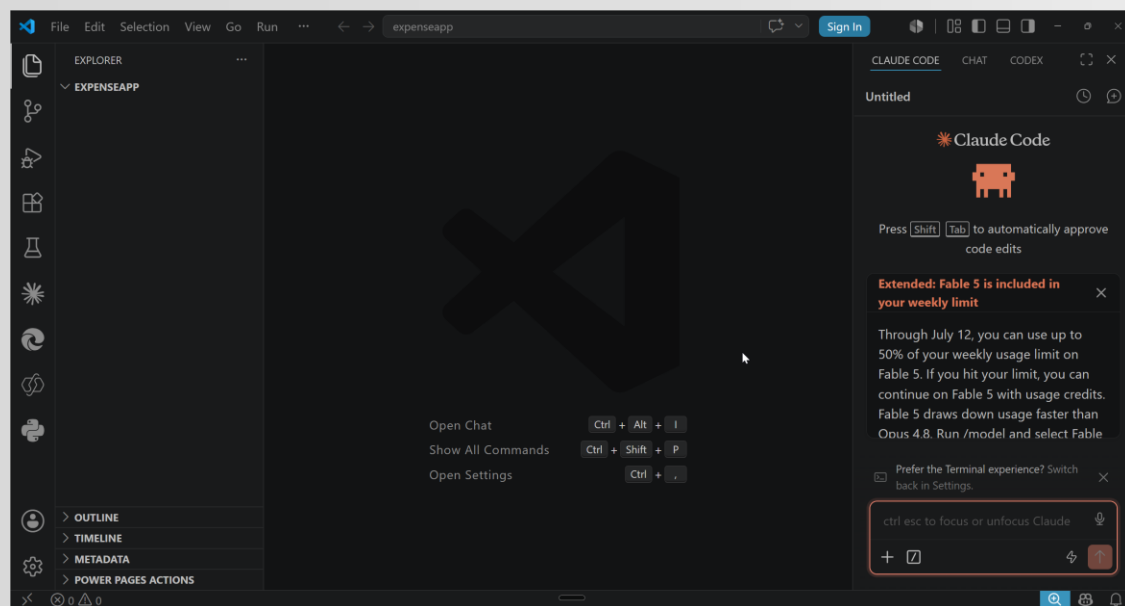
THREE EXTENSIONS TO START



OPEN A PROJECT FOLDER



SIDE PANEL OR TERMINAL



PICK THE HARNESS YOU LIKE

Claude Code

- Terminal or Visual Studio Code, on an Anthropic subscription. Driving the demo today.

Codex

- OpenAI's agent, in the editor or the terminal. Same loop, same shape of work.

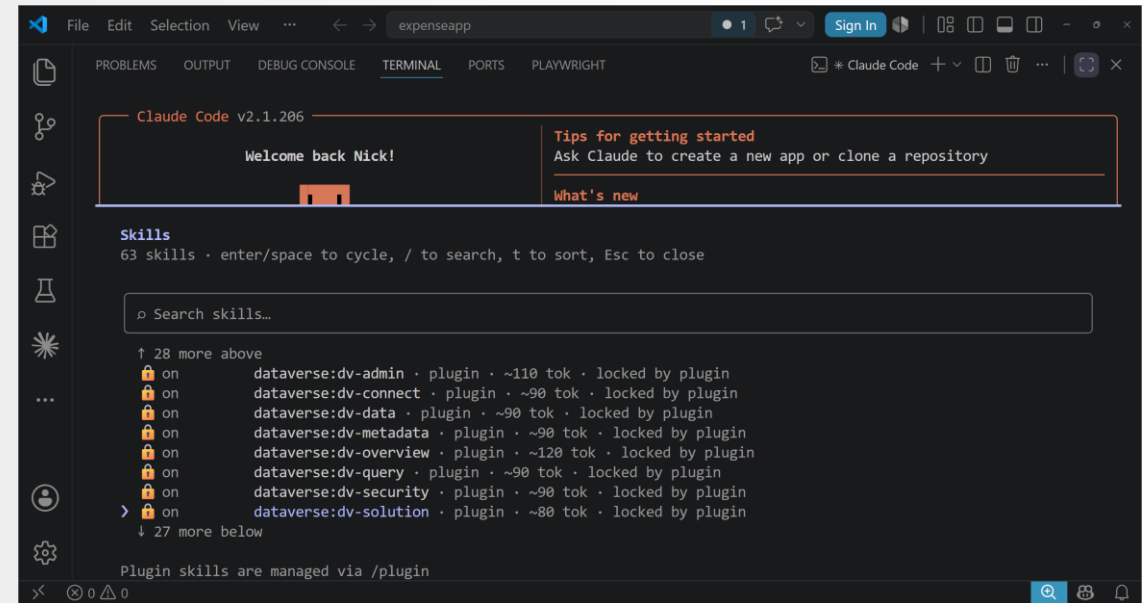
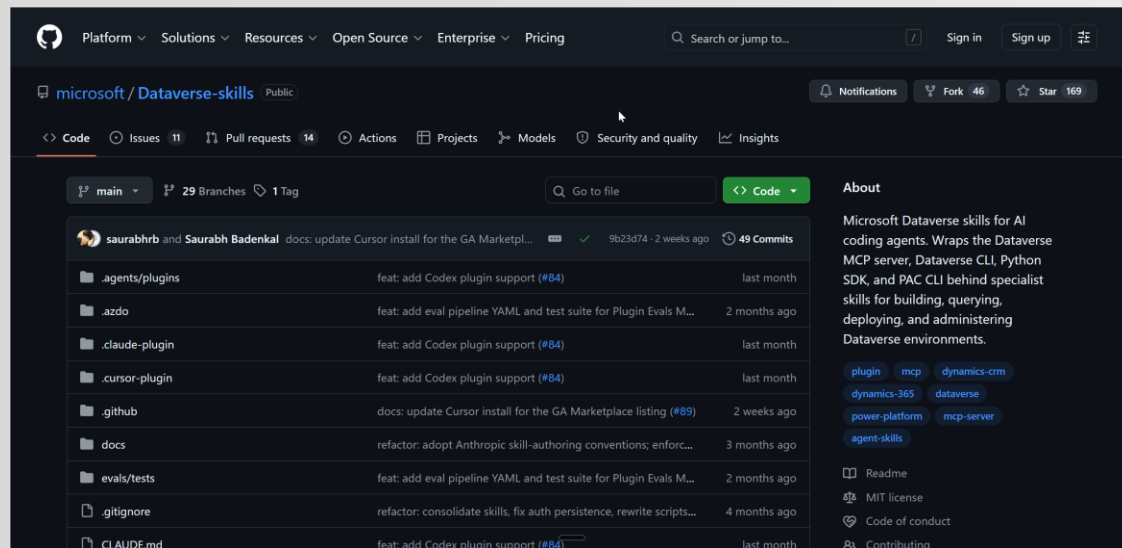
GitHub Copilot

- Same workflow, same skills, inside Visual Studio Code on a GitHub subscription.

The point

- Harnesses and models change every few months. The workflow does not. That is what to learn.

INSTALL THE DATAVERSE SKILLS



WHAT THE SKILLS WRAP

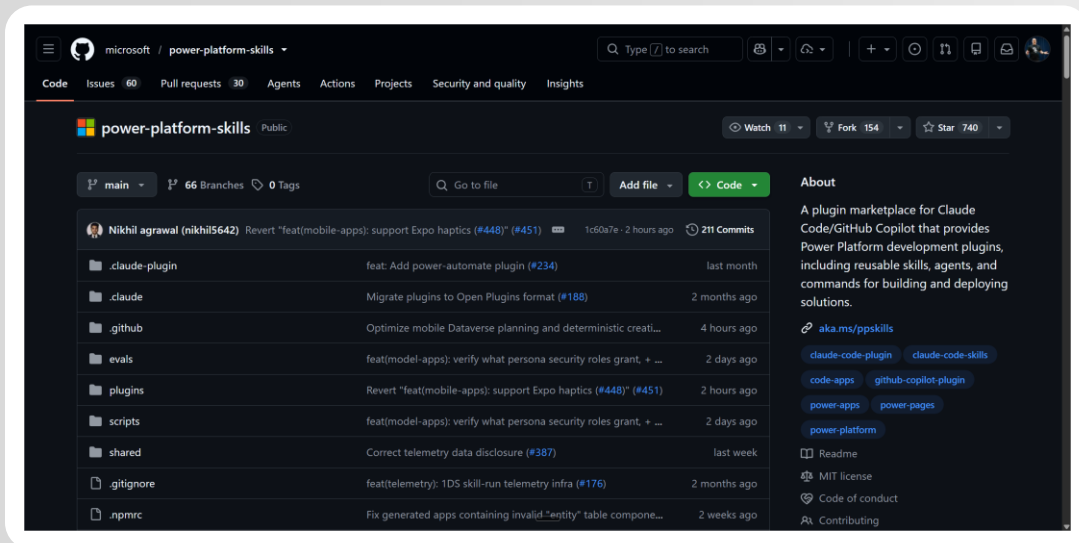
One install, three tools

- microsoft/Dataverse-skills on GitHub.
- Wraps the Dataverse MCP server, the Dataverse CLI and the PAC CLI.
- Install commands for Claude Code and GitHub Copilot are both in the README.

What lands in your project

- dv-connect, dv-metadata, dv-solution, dv-data, dv-query, dv-security, dv-admin and more.
- An .env file with your environment URL, tenant ID and the MCP client ID it enabled for you.
- Microsoft Learn and Dataverse MCP servers, wired in.

AND THE POWER PLATFORM SKILLS



microsoft/power-platform-skills. One installer script.

What they build

Generative pages, model-driven and Canvas apps, flows, Power Pages code sites.

Where to get them

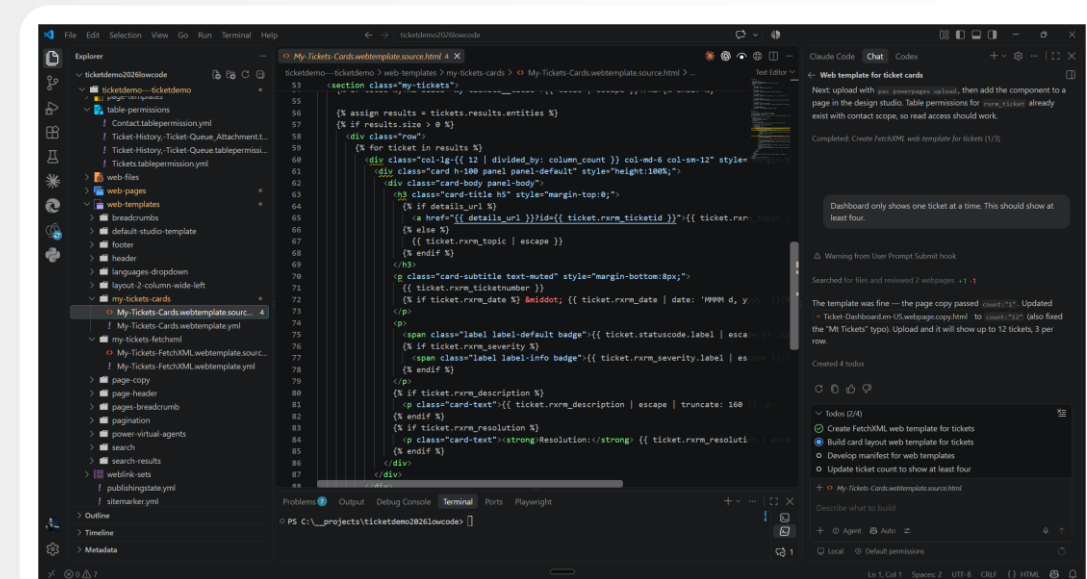
github.com/microsoft/power-platform-skills.
One script, then reload.

The gap today

No skill yet for traditional low-code Power Pages sites.

You are not stuck

Liquid, JavaScript, the Web API, web pages and web templates.

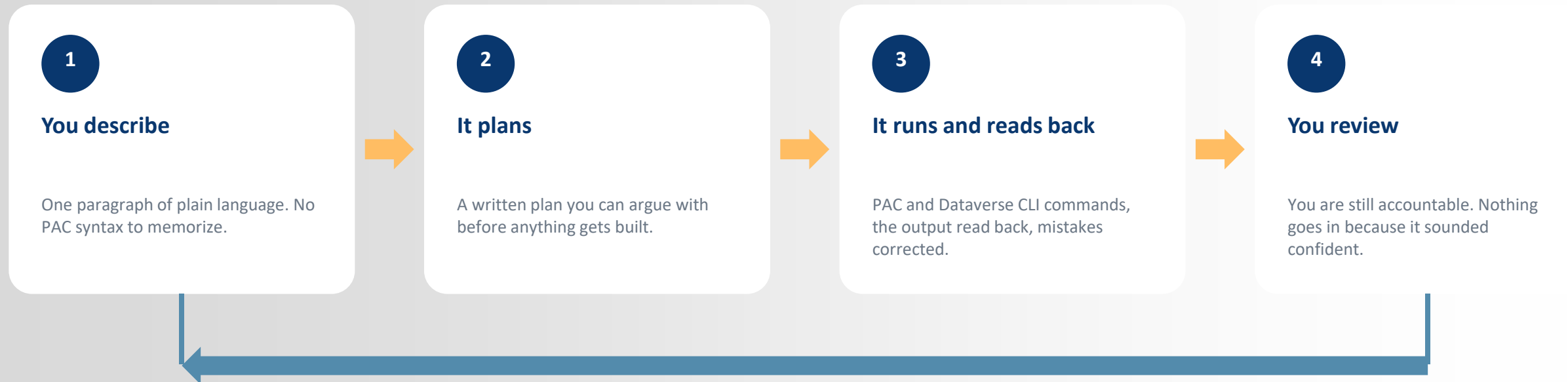


Web templates and Liquid still respond well to an agent.

AGENTIC, NOT AUTOCOMPLETE



THE LOOP IS THE POINT



Then round again, with what the last lap taught it. That second lap is where the quality comes from.

LET'S BUILD IT



#BalticSummit2026

THE DEMO, IN FIVE STEPS

- Environment, from one prompt
- Auth profile, workspace, MCP server
- Solution and publisher, prefix tkt
- Tables, relations and sample data
- A model-driven app on top

#GrowBetterTogether

Unleash the power of Microsoft solutions

THREE HABITS THAT PAY YOU BACK

01

Write the instructions down

Run /init once and keep a CLAUDE.md in the project. Prefixes, conventions, the things you do not want to explain twice. The agent reads it every session.

02

Ask for a changelog

Have it append to changelog.md as it works. It turns a black box into a reviewable diff, and it is the only record of why something was built that way.

03

Plan first, keep the prompt

Use the plan skill before you build. When it works, ask the agent to write the prompt it wishes it had on day one. That file is the artifact, not the code.

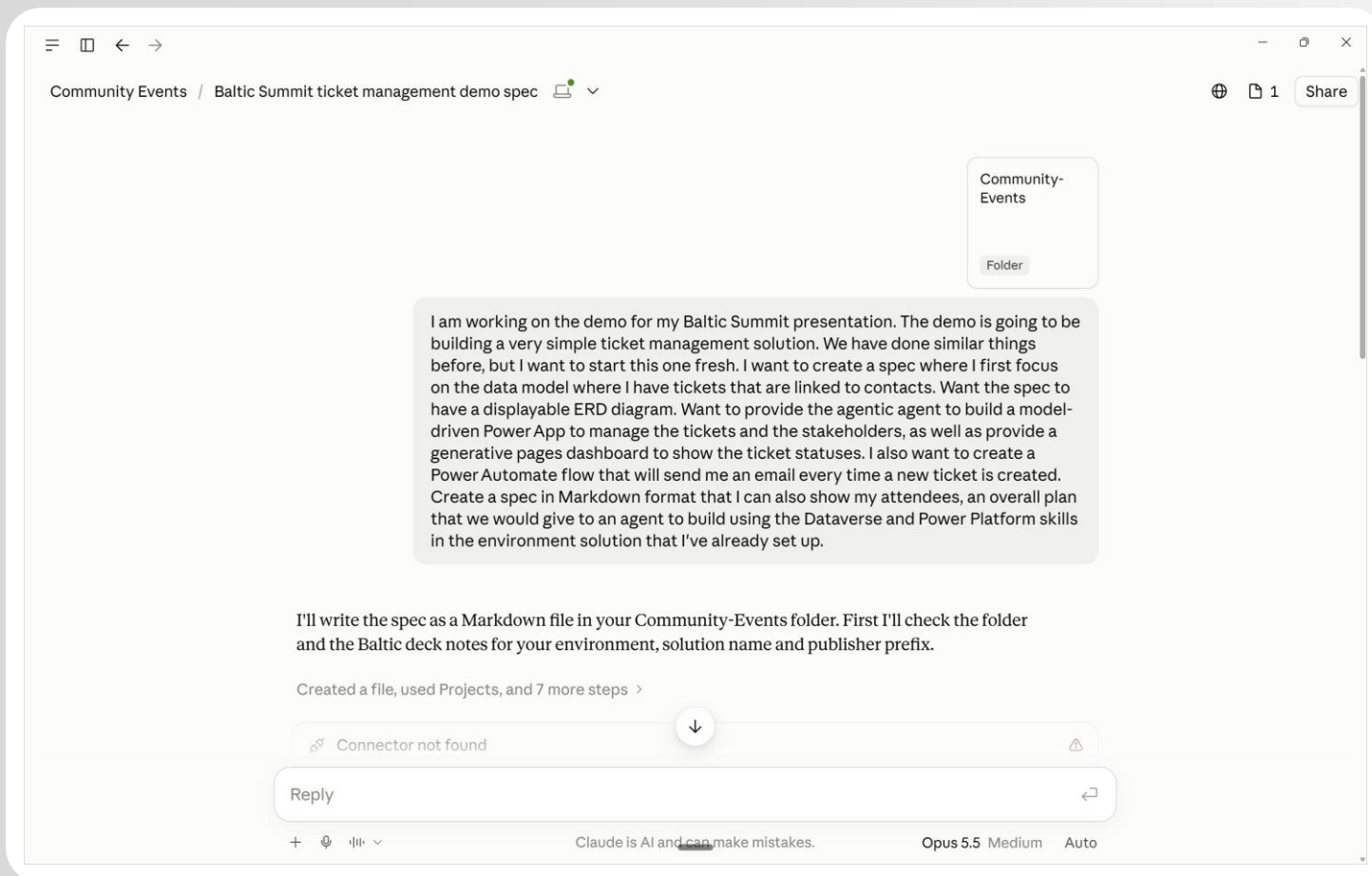
*Who here has always wanted to go back and rebuild something they already shipped?
The code was always the cheap part. Keep the spec and the rebuild is an afternoon.*

Slides and
the project



[github.com/readyxrm/
balticsummit2026](https://github.com/readyxrm/balticsummit2026)

STEP 1: COLLECT THE REQUIREMENTS



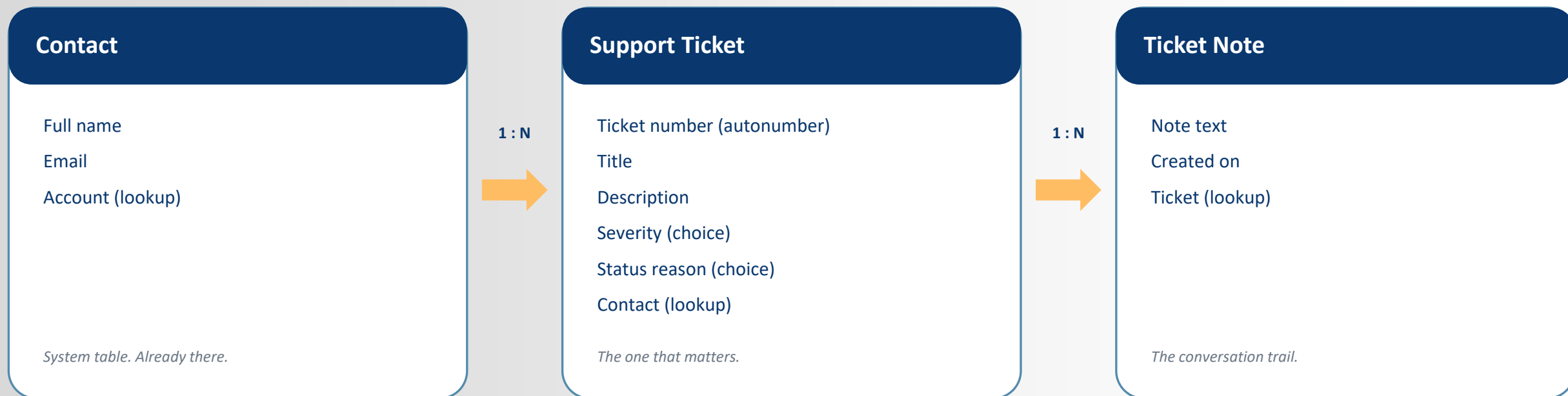
FOR REFERENCE

Describe the solution in plain language to a general purpose agent. Data model first, then the app, the dashboard and the flow.

Ask it to write the spec as markdown you can put in front of people.

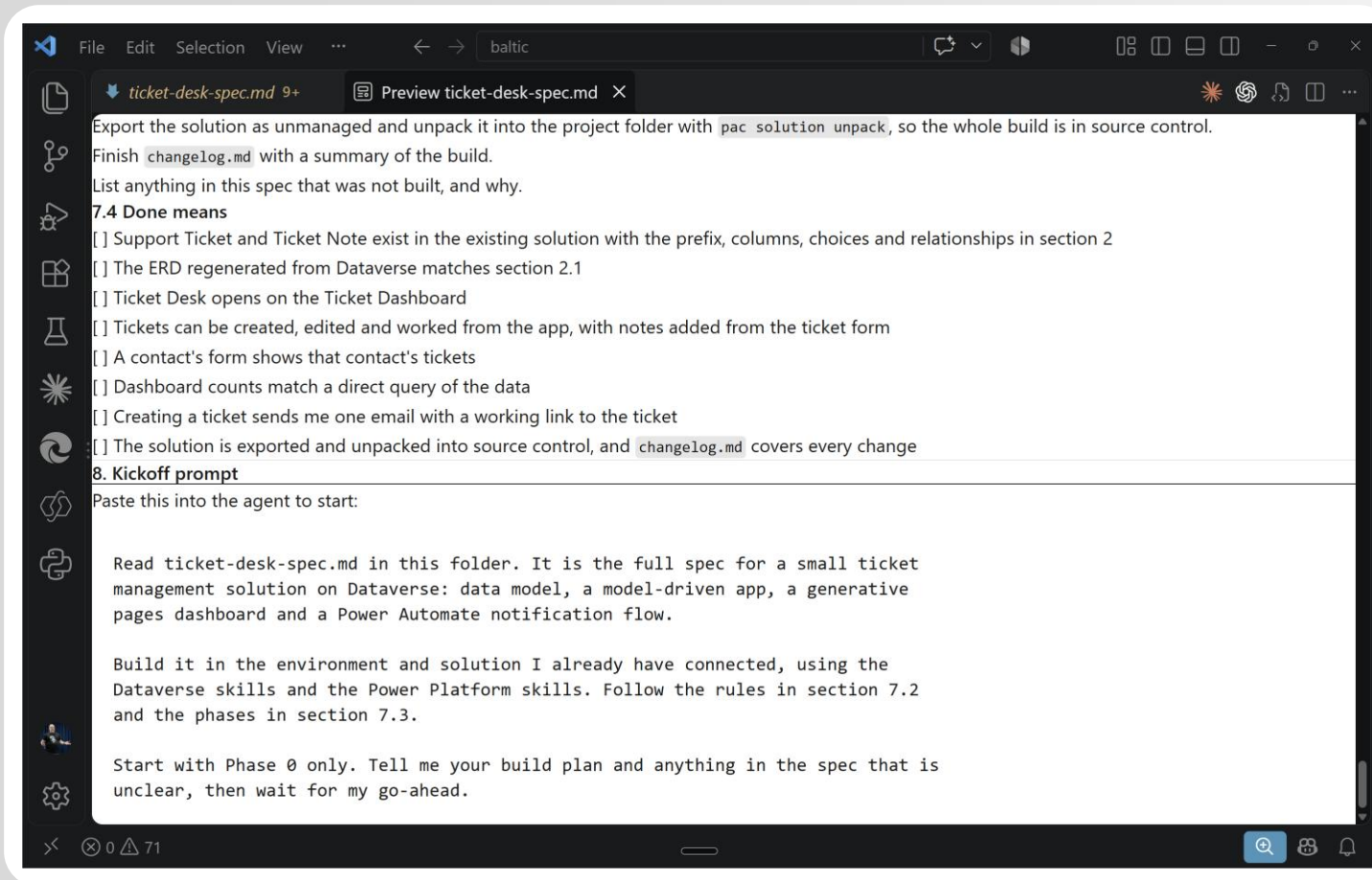
Spec written in a Claude project, not in the editor. No code involved yet.

THE MODEL I DREW FIRST



Three tables, two relationships. Ten minutes on paper, before the agent writes anything.

STEP 2: THE SPEC IT WROTE



```
File Edit Selection View ... ← → baltic
ticket-desk-spec.md 9+ Preview ticket-desk-spec.md X
Export the solution as unmanaged and unpack it into the project folder with pac solution unpack, so the whole build is in source control.
Finish changelog.md with a summary of the build.
List anything in this spec that was not built, and why.
7.4 Done means
[ ] Support Ticket and Ticket Note exist in the existing solution with the prefix, columns, choices and relationships in section 2
[ ] The ERD regenerated from Dataverse matches section 2.1
[ ] Ticket Desk opens on the Ticket Dashboard
[ ] Tickets can be created, edited and worked from the app, with notes added from the ticket form
[ ] A contact's form shows that contact's tickets
[ ] Dashboard counts match a direct query of the data
[ ] Creating a ticket sends me one email with a working link to the ticket
[ ] The solution is exported and unpacked into source control, and changelog.md covers every change
8. Kickoff prompt
Paste this into the agent to start:

Read ticket-desk-spec.md in this folder. It is the full spec for a small ticket management solution on Dataverse: data model, a model-driven app, a generative pages dashboard and a Power Automate notification flow.

Build it in the environment and solution I already have connected, using the Dataverse skills and the Power Platform skills. Follow the rules in section 7.2 and the phases in section 7.3.

Start with Phase 0 only. Tell me your build plan and anything in the spec that is unclear, then wait for my go-ahead.
```

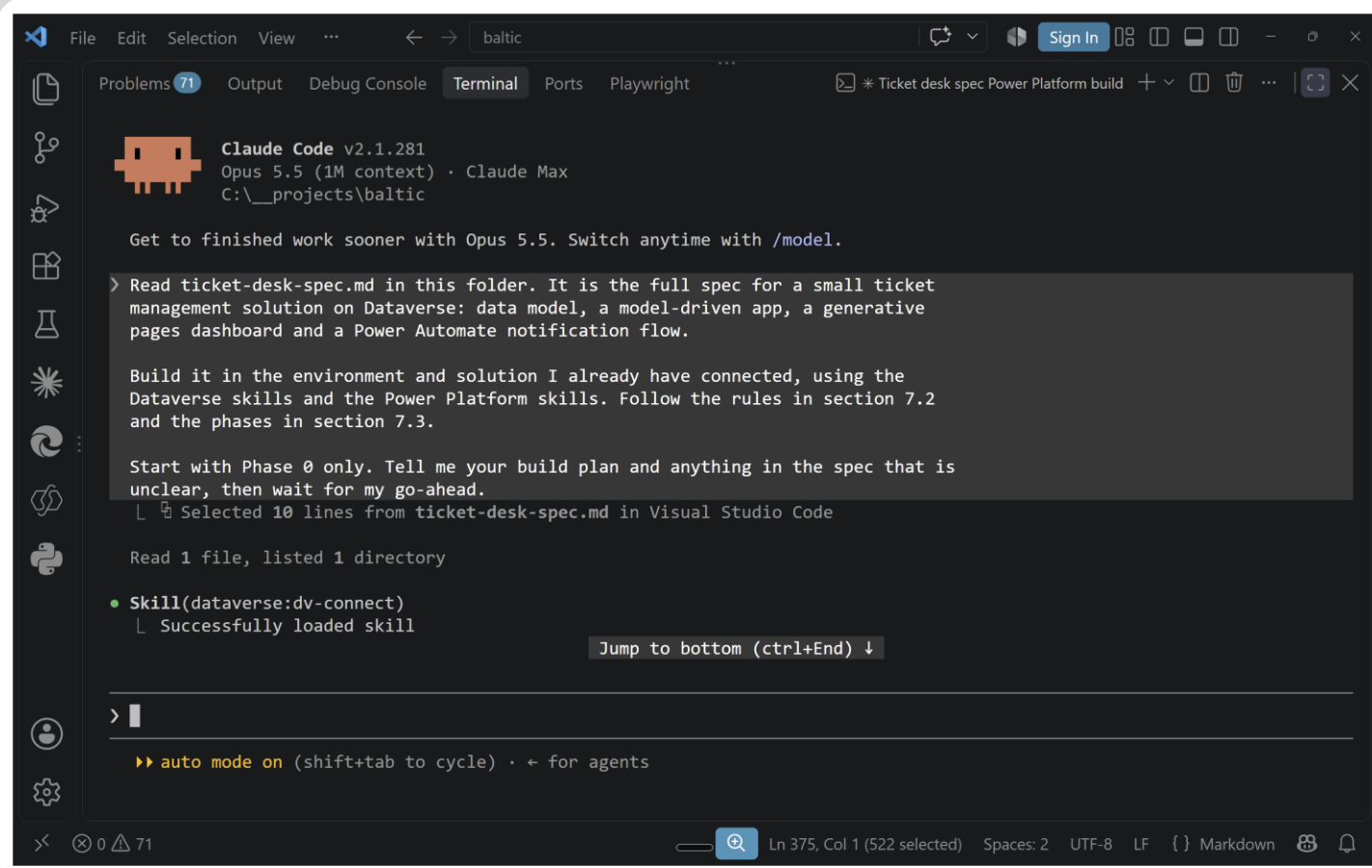
FOR REFERENCE

One markdown file. Tables, columns, relationships, an ERD, the build phases, a done-means checklist and a kickoff prompt at the end.

This is the page you review. Not ten thousand lines of generated code.

[ticket-desk-spec.md](#), open in Visual Studio Code.

STEP 3: HAND IT TO THE AGENT



```
File Edit Selection View ... ← → baltic Sign In
Problems 71 Output Debug Console Terminal Ports Playwright
* Ticket desk spec Power Platform build + - ...

Claude Code v2.1.281
Opus 5.5 (1M context) · Claude Max
C:\__projects\baltic

Get to finished work sooner with Opus 5.5. Switch anytime with /model.

> Read ticket-desk-spec.md in this folder. It is the full spec for a small ticket
management solution on Dataverse: data model, a model-driven app, a generative
pages dashboard and a Power Automate notification flow.

Build it in the environment and solution I already have connected, using the
Dataverse skills and the Power Platform skills. Follow the rules in section 7.2
and the phases in section 7.3.

Start with Phase 0 only. Tell me your build plan and anything in the spec that is
unclear, then wait for my go-ahead.
  Selected 10 lines from ticket-desk-spec.md in Visual Studio Code

Read 1 file, listed 1 directory

• Skill(dataverse:dv-connect)
  Successfully loaded skill

Jump to bottom (ctrl+End) ↓

>

>> auto mode on (shift+tab to cycle) · ← for agents
```

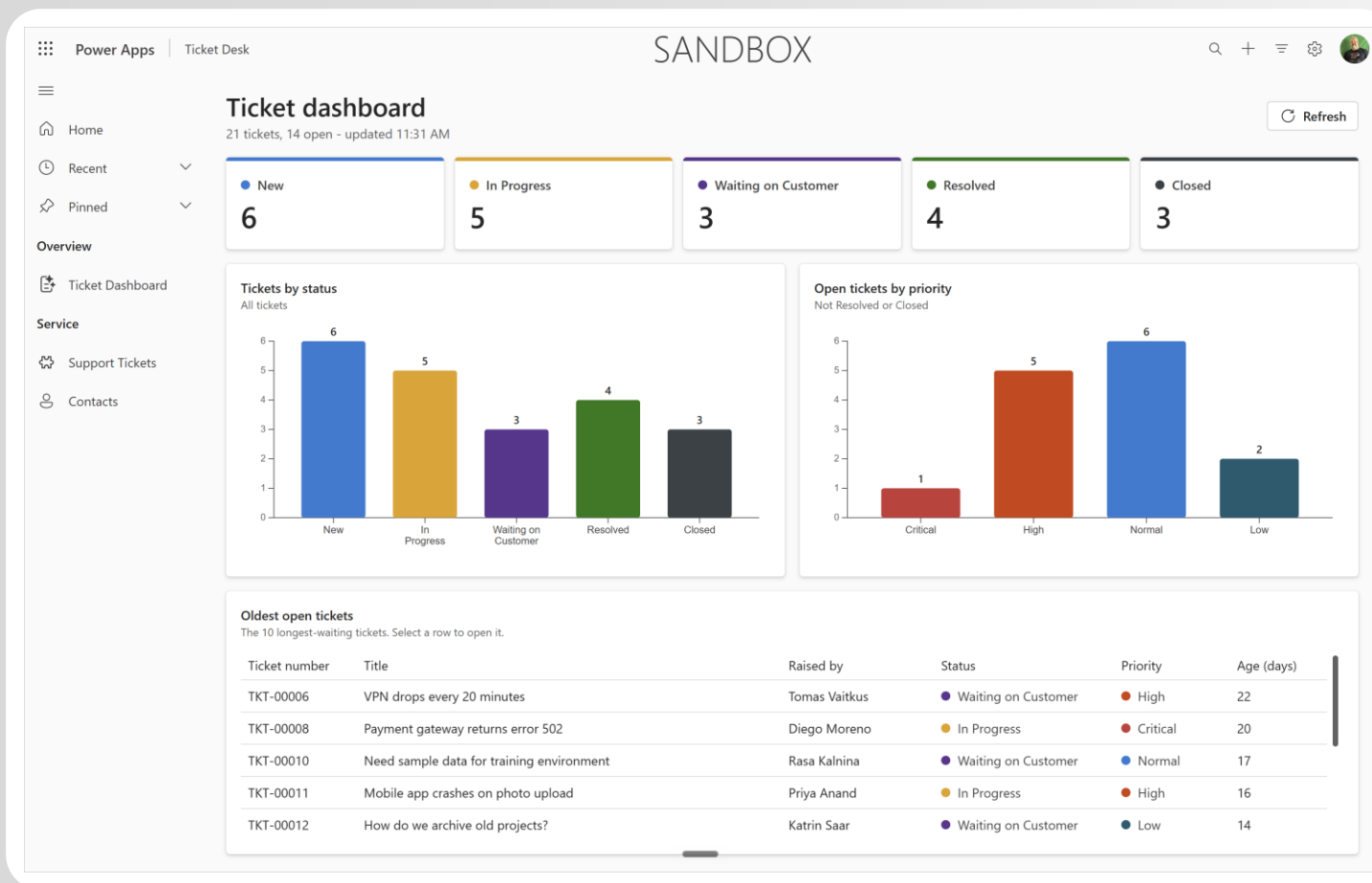
FOR REFERENCE

Paste the kickoff prompt into Claude Code in the Visual Studio Code terminal. It reads the spec and loads the Dataverse skills.

Phase 0 only. It comes back with a build plan and its questions before it touches the environment.

The prompt came out of the spec. Nothing was retyped.

STEP 4: WHAT CAME OUT



FOR REFERENCE

Ticket Desk, a model-driven app with a generative pages dashboard on the Dataverse tables from the spec.

Status counts, open tickets by priority, and the oldest open tickets in a list you can click through.

Built from one spec and one prompt, in the environment we set up earlier.

WHERE IT BITES



WATCH FOR THESE

It fills in your blanks

- I asked for amounts and got decimal columns instead of currency. It caught it and reworked. Say it up front.

Security is still yours

- Table permissions and roles do not get a pass because an agent wrote them. Check them yourself.

Approve every command

- Early on, watch what it runs so you learn the process. Automate it later, not first.

It is keen

- It will happily build the whole app before you asked. One step at a time until you trust it.

“

This is a tool. It is not a replacement for knowing your craft.

NICK DOELMAN
BAL TIC SUMMIT 2026



DO THIS ON MONDAY

Install three things

- Visual Studio Code, Power Platform Tools, and the agent you already pay for.

Draw the model first

- Tables, columns, choices, relationships. On paper. Then hand it over.

Install the skills

- One command from the microsoft/Dataverse-skills repo. Then ask it to list your tables.

Build something real

- A ticketing app, an expense app, anything with a user. An hour of this beats any deck.

TAKE THIS WITH YOU

Read and watch

- readyxrm.blog/2026/07/16/ditch-the-power-apps-maker-portal/
- youtu.be/uGgeGO5TMA4
- github.com/microsoft/Dataverse-skills
- github.com/microsoft/power-platform-skills

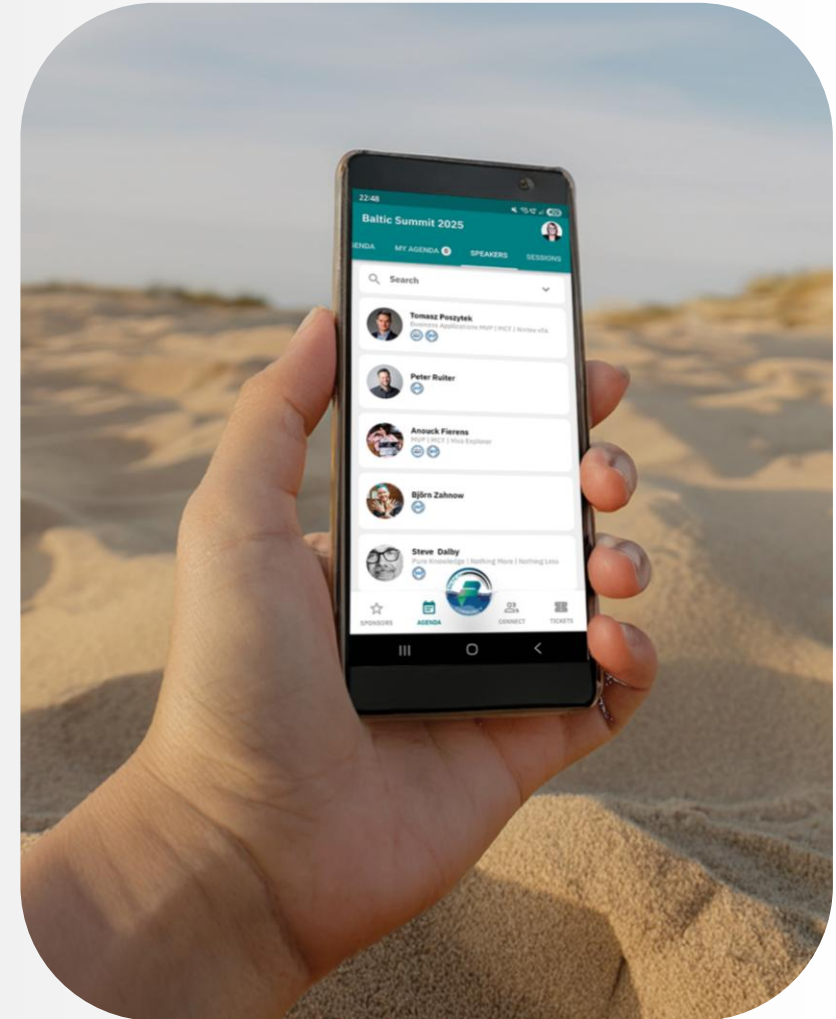
Code and contact

- github.com/readyxrm/balticsummit2026
- nickdoelman.com
- readyxrm.blog
- [#BalticSummit2026](https://twitter.com/BalticSummit2026)

QUESTIONS?

**PLEASE RATE THIS
SESSION IN THE APP**

run_oevents



THANK YOU FOR PARTICIPATION

nickdoelman.com

readyxrm.blog

[#BalticSummit2026](https://twitter.com/BalticSummit2026)

SPONSORS



COMMUNITIES & PARTNERS

